

I. Dataset Format

First, I began by considering the format of the input data. Each handwritten digit image is originally a 28×28 pixel grid, but for the purpose of building a neural network, I treated it as a one-dimensional array of 784 values. Fortunately, I did not need to manually convert the data, as I was able to use a CSV dataset provided by a respected contributor on Kaggle, which made the preprocessing step straightforward.

<https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

II. Load MNIST from CSV

The function `load_mnist_inCSV` loads the MNIST dataset from a CSV file, shuffles the data, normalizes the pixel values to the range $[0, 1]$, and separates the labels from the input features. Each image is represented as a 784-dimensional column vector, while the corresponding labels are stored as a one-dimensional array.

III. Parameters Initialization

The weights and biases of each layer are generated using the `random.randn` function provided by NumPy. The first version initialized the weights by subtracting 0.5 from each randomly generated value to avoid potential hidden parameter issues. The second version references this article:

<https://www.kaggle.com/discussions/general/299381>

Replace the first version by “He initialization” method, each generated weight is multiplied by $\sqrt{2/\text{weights of layer}}$.

IV. Neural Network

This Neural Network will have a simple two-layer architecture. Input layer $a[0]$ will have 784 units corresponding to the 784 pixels in each 28×28 input image. A hidden layer $a[1]$ will have 10 units with ReLU activation, and finally our output layer $a[2]$ will have 10 units corresponding to the each of ten classes (0–9) with softmax activation.

V. Backward Propagation

* ReLU Derivative : $(Z > 0).astype(float)$.

Converting to float is important; otherwise, gradient vanishing will occur, and without it, the phenomenon of no updates will be clearly observed.

* Output Layer Softmax Derivative :

Predict the probability distribution minus the one-hot encoded Y .

* Derivative of Weighted Layer : $1/m * \text{Dot product of derivatives from the previous layer to the input matrix of this layer.}$

* Derivative of Bias Layer : $1/m * \text{sum of derivatives from the previous layer.}$

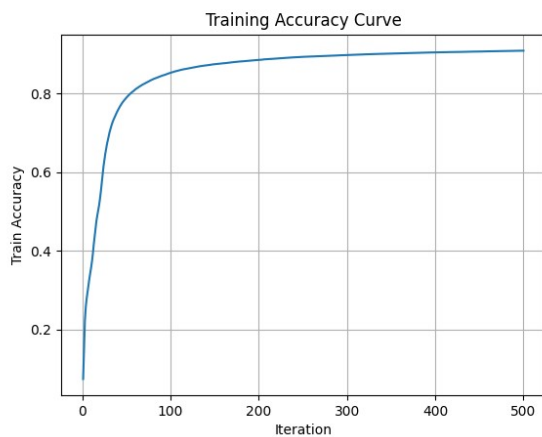
VI. Update Parameters

For all new weight layers and new bias layers will be original themselves minus alpha parameter times the corresponding layer's derivative. $\text{Alpha} = 0.1$

VII. One iteration

For each iteration, the network performs forward propagation to compute the activations, followed by backward propagation to calculate the gradients of the loss with respect to the parameters. These gradients are then used to update the weights and biases through gradient descent, gradually improving the model's performance.

VIII. Result Original



After 500 times training it could reach **90.91 %** Accuracy on Training Dataset and reach **91.13 %** on Test Dataset.

IX. Engineering Optimization

The second version completely replaced the code with an object-oriented design method. I'm pretty sure this approach makes data like weights much easier to manage, eliminating the problems of variable propagation errors that often occur when first writing tests. The overall code looks better .

X. Reference

- <https://youtu.be/VMj-3S1tku0?si=rLMQOCWUYGOOLZx0>
- <https://medium.com/@ombaval/building-a-simple-neural-network-from-scratch-for-mnist-digit-recognition-without-using-7005a7733418>
- Deep Learning Theory and Practice Course in National Kaohsiung University of Science and Technology

XI. Adam Optimizer for NeuralNet

This part uses the original architecture, which has 10 Neuron cores in the hidden layer. An experiment was conducted to modify the original gradient descent optimizer to Adam Optimizer which reference from Adam: A Method for Stochastic Optimization,2014.

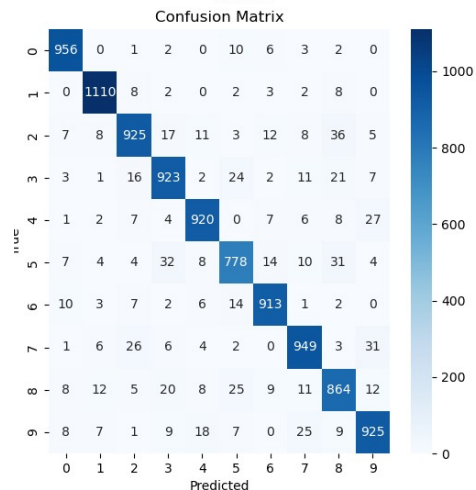
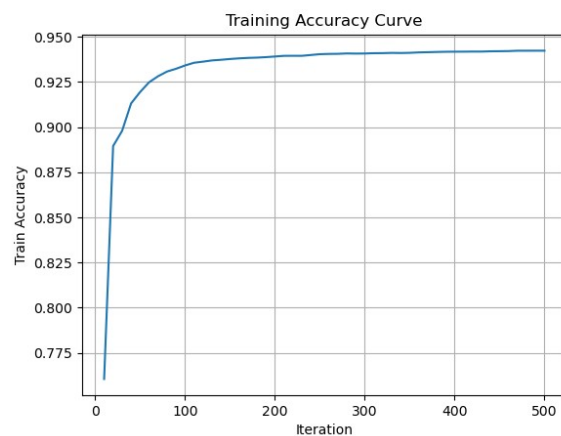
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v} = \frac{v_t}{1 - \beta_2^t}$$

$$\theta = \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

After experiment of implement Adam Optimizer and replace original gradient descent algorithm. The accuracy improved from 91.13% to **.92.63%** on Test Dataset, and 94.23% on Training Dataset.



XII. Fox Optimization Algorithm for Hyperparameters Search

(1) Background: Parameter Optimization and Metaheuristics in Neural Networks

Parameter optimization has been a long been an import topic in machine learning. Early neural networks mainly relied on gradient-based optimization methods. However, hyperparameter tuning such like learning rate and architecture size, often requires searching over complex and non-convex spaces. Metaheuristic algorithms, including evolutionary algorithms, particle swarm optimization, and genetic algorithms, have therefore been explored as alternative strategies for neural network parameter search.

(2) Motivation: From Evolutionary Computation to FOA + MLP

The idea for this experiment originated from my academic experience. During my sophomore year, I took a course on evolutionary computation, where various metaheuristic optimization algorithms were introduced. After the course, I became interested in exploring other metaheuristic approaches beyond the class material. In recent literature (around 2023-2025), I encountered a relatively new optimization method known as the Fox Optimization Algorithm (FOA). Inspired by this, I proposed a simple experimental idea: applying FOA to neural network hyperparameter tuning. In this project, FOA was used to search for suitable hyperparameters of a multilayer perceptron (MLP), specifically the learning rate and hidden layer size.

(3) Method: Fox Optimization Algorithm for Hyperparameter Search

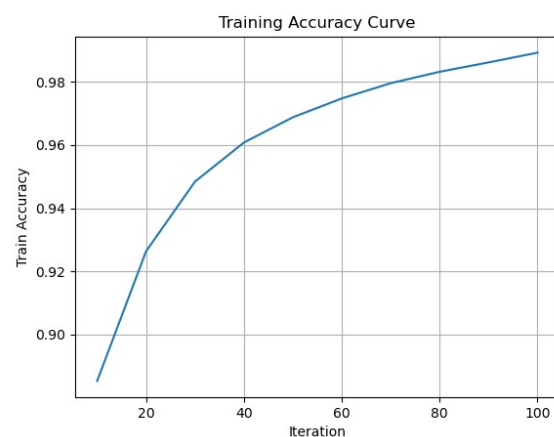
Excerpt from the paper: FOX: A Fox-inspired Optimization Algorithm

The Fox Optimization Algorithm is a population-based metaheuristic inspired by the hunting behavior of foxes. Each candidate solution represents a point in the hyperparameter search space. The algorithm iteratively updates

candidate solutions by combining exploitation (moving toward the current best solution) and exploration (random perturbations and interactions with other candidates). In this work, each individual in the population represents two hyperparameters: the learning rate and the number of hidden units in the MLP. During each iteration, candidate solutions are updated by three components: attraction toward the current best solution, stochastic exploration noise, and interaction with randomly selected individuals. After generating a new candidate, the model is trained for a short number of iterations on a subset of the dataset, and its classification accuracy is used as the fitness score. A greedy selection strategy keeps improved candidates while discarding worse ones.

(4) Results and Implications for Deep Learning

Search target set one hidden layer and aim to search the best hyperparameter of hidden layer percetron units and learning rate. After 5 iteration (generation) of Fox Optimization and population set 8 foxes. In fact, it could get a very good result by just search 1 iteration. The final number of hidden layer unit is 405, and learning rate is 0.01. For training 100 epochs. In the end, it reach **97.53% on Test Dataset**, and 98.93% on training dataset.



The experiment demonstrates that FOA can be applied as a hyperparameter search method for neural networks. By treating hyperparameter selection as a global optimization problem, FOA

provides a simple and flexible way to explore the search space without requiring gradient information. Although this experiment is relatively small-scale, it illustrates how metaheuristic optimization can complement gradient-based training in deep learning pipelines. Such approaches may be particularly useful for neural architecture search or automated hyperparameter tuning, where the optimization landscape is complex and highly non-linear.

